

Migrate Legacy SocialUser Module to New SocialUser-Service Microservice #112



daironpf

opened 35 minutes ago

Owner

...

This issue covers the full migration of the legacy **SocialUser** module from the old monolithic/legacy codebase into the new dedicated **SocialUser-Service microservice**, aligned with the updated microservice architecture and domain boundaries of SocialSeed.

The new SocialUser-Service will be responsible *only* for managing user social identity data (public profile, display name, bio, avatar, settings, etc.) and will no longer contain any relationship graph logic. All relationship-related features (followers, friends, mutual connections, etc.) will be handled by the future **Relationships-Service**.

This migration includes reviewing old data models, extracting reusable logic, adapting it to hexagonal architecture (domain → application → infrastructure), refactoring DTOs, ports, and repositories, and ensuring the new service is clean, independent, and fully aligned with the new Auth–SocialUser boundaries.

Scope of Work

1. Extract old SocialUser domain logic

- Identify all legacy SocialUser models, services, and controllers.
- Classify reusable parts vs. deprecated functionality.
- Remove relationship-based logic (to be migrated later into Relationships-Service).

2. Create new SocialUser-Service structure

- Set up hexagonal architecture folders.
- Define domain models: `SocialUser`, `ProfileSettings`, `VisibilityOptions`, etc.
- Define application use cases: `GetUser`, `UpdateUser`, `CreateSocialUserNode`.

3. Implement gRPC adapters

- Add incoming gRPC interface for Auth-Service integration (`CreateSocialNode`, `SyncUserData`).
- Add outgoing gRPC client for future interactions.

4. Migrate and refactor persistence layer

- Map old fields → new clean schema.
- Implement repository ports + adapters.
- Add Neo4j indexing optimizations.
- Validate userId consistency rules.

5. Implement DTOs and mappers

- Replace legacy DTOs with new standardized DTO models.
- Remove obsolete fields.

6. Ensure compatibility with Auth-Service

- Validate all registration + sync workflows.
- Ensure SocialUser nodes are correctly created at registration via gRPC.

7. Write integration and unit tests

- Domain tests
- Use-case tests
- Infrastructure adapter tests
- gRPC contract tests

8. Documentation

- Update `/docs/architecture/socialuser/`
- Add migration notes
- Add new API contract and gRPC definitions outline

Acceptance Criteria

- ☐ All legacy SocialUser logic is analyzed and documented.
- ☐ New SocialUser-Service structure is fully created following hexagonal architecture.
- ☐ All reusable logic is migrated and refactored into the new domain model.
- ☐ Relationship-related code is removed and isolated for future Relationships-Service.
- ☐ gRPC interface for Auth-Service is fully implemented and tested.
- ☐ Persistence layer for Neo4j is fully migrated, validated, and tested.
- ☐ End-to-end flow from Auth-Service → SocialUser-Service works successfully.
- ☐ Unit, integration, and contract tests are completed and passing.
- ☐ Documentation is updated and reflects the new service boundaries and architecture.
- ☐ The new service is stable, standalone, and ready for deployment.

Create sub-issue

😊

- daironpf self-assigned this 35 minutes ago
- daironpf moved this to In Progress in

SocialSeed

 35 minutes ago
- daironpf added this to

SocialSeed

 35 minutes ago
- daironpf added

architecture

refactor

priority-high

cleanup

migration

 35 minutes ago
- daironpf pinned this issue now



Add a comment

Write

Preview

H B I | | <> | | | | | @ | | | |

Use Markdown to format your comment

Paste, drop, or click to add files

Close issue

Comment

Assignees

daironpf

Labels

migration refactor priority-high architecture cleanup

Projects

SocialSeed

Status In Progress

Labels

migration refactor priority-high architecture cleanup

Projects

SocialSeed

Status In Progress

Milestone

No milestone

Relationships

None yet

Development

112-migrate-legacy-socialuser-module-to-new-socialuser-service-microservice